1a. Write a shell script that takes a valid directory name as an argument and recursively descends all the sub-directories, finds the maximum length of any file in that hierarchy and writes this maximum value to the standard output.

1a.sh

```
# Multiple Arguments
for i in "$@"
do
    if [ -d "$i" ]
    then
           echo "Large file size is:"
           ls -Rl "$i" | grep "^-" | tr -s ' ' | cut -d ' ' -f 5,9 |
           sort -n | tail -1
     else
            echo "Not a directory"
     fi
done
```

```
student@student:~/Unix$ sh 1a.sh /home/student/Downloads
 /home/student/Documents
 Large file size is:
 373575296 DesktopEditors_x64.exe
 Large file size is:
 16024 Student_List.txt
```

1aa.sh

```
# Single Argument
if [ -d "$1" ]
then
       echo "Large file size is:"
       ls -Rl "$1" | grep "^-" | tr -s ' ' | cut -d ' ' -f 5,9 |
        sort -n | tail -1
else
       echo "Not a directory"
fi
```

```
student@student:~/Unix$ sh 1aa.sh /home/student/Downloads
 Large file size is:
 373575296 DesktopEditors_x64.exe
```

1b. Write a shell script that accepts a path name and creates all the components in that path name as directories. For example, if the script is named mpc, then the command mpc a/b/c/d should create directories a, a/b, a/b/c, a/b/c/d.

1b.sh

```
if [ $# -eq 0 ]
then
    echo "No arguments"
else
      curdir=`pwd`

      for dir in `echo $1 | tr '/' ' ' `
      do
          if [ -d $dir ]
          then
                  echo "$dir exists under `pwd`"
          else
                  mkdir $dir
                  echo "$dir created under `pwd`"
          fi
          cd $dir
      done
      cd $curdir
fi
```

```
student@student:~/Unix$ sh 1b.sh a/b
 a created under /home/student/Unix
 b created under /home/student/Unix/a
student@student:~/Unix$ sh 1b.sh a/b/c/d
 a exists under /home/student/Unix
 b exists under /home/student/Unix/a
 c created under /home/student/Unix/a/b
 d created under /home/student/Unix/a/b/c
```

2a. Write a shell script that accepts two filenames as arguments, checks if the permissions for these files are identical and if the permissions are identical, output common permissions and otherwise output each filename followed by its permissions.

2a.sh

```
if [ $# -ne 2 ]
then
    echo "Enter two filenames"
else
    p1=`ls -l "$1" | cut -c 2-10`
    p2=`ls -l "$2" | cut -c 2-10`

    if [ "$p1" = "$p2" ]
    then
        echo "Common permissions: "
    else
        echo "Different permissions:"
    fi
    echo "$1 : $p1"
    echo "$2 : $p2"
fi
```

```
student@student:~/Unix$ touch a1 a2
student@student:~/Unix$ sh 2a.sh a1 a2
 Common permissions:
 a1 : rw-rw-r-
 a2 : rw-rw-r-
student@student:~/Unix$ chmod 755 a1
student@student:~/Unix$ chmod u+x a2
student@student:~/Unix$ sh 2a.sh a1 a2
 Different permissions:
 a1 : rwxr-xr-x
 a2 : rwxrw-r--
```

2b. Write a shell script which accepts valid log-in names as arguments and prints their corresponding home directories, if no arguments are specified, print a suitable error message.

2b.sh

```
if [ $# -eq 0 ]
then
    echo "No login names specified"
else

    for user in "$@"
    do
            home=`grep "^$user:" /etc/passwd | cut -d : -f 6`
            if [  "$home" != "" ]
            then
                 echo "$user : $home"
            else
                 echo "$user : Invalid login name"
            fi
    done
fi
```

```
student@student:~/Unix$  sh 2b.sh  student nmamit shree root
 student : /home/student
 nmamit :  /home/nmamit
 shree : Invalid login name
 root : /root
```

3a. Create a script file called file-properties that reads a filename entered and outputs it Properties.

3a.sh

```
echo "Enter the file name"
read file

if [ -f "$file" ]
then
    set -- ` ls -l "$file" `

    echo "File permission : $1"
    echo "Number of links  : $2"
    echo "User name        : $3"
    echo "Group name       : $4"
    echo "File size        : $5 bytes"
    echo "Date of modification : $6 $7"
    echo "Time of modification : $8"
    echo "Name of file     : $9"
    echo "Type : `file -b "$file"`"
    ls -l "$file"
else
    echo "File does not exist"
fi
```

```
student@student:~/Unix$  sh 3a.sh
 Enter the file name
 2b.sh
 File permission : -rw-rw-r--
 Number of links  : 1
 User name        : student
 Group name       : student
 File size        : 279 bytes
 Date of modification : Aug 18
 Time of modification : 20:27
 Name of file     : 2b.sh
 Type : ASCII text
 -rw-rw-r-- 1 student student 279 Aug 25 20:27 2b.sh
```

3b. Write a shell script to implement terminal locking (similar to the lock command).It should prompt the user for a password. After accepting the password entered by the user, it must prompt again for the matching password as confirmation and If a match occurs, it must lock the keyword until a matching Password is entered again by the user, Note that the script must be written to disregard BREAK, control-D. No time limit needs to be implemented for the lock duration.

3b.sh

```
clear

echo "Enter password:"
stty -echo
read pass1
stty echo

echo "Confirm password:"
stty -echo
read pass2
stty echo

while [ "$pass1" != "$pass2" ]
do
        echo "Password does not match"
         echo "Confirm password again:"
         stty -echo
         read pass2
         stty echo
done

trap '' 2 3
echo "Terminal is locked"
echo "Enter password to unlock:"

stty -echo
read pass3
stty echo

while [ "$pass3" != "$pass1" ]
do
         echo
         echo "Invalid password"
         echo "Enter password again:"
         stty -echo
         read pass3
         stty echo
done

echo
echo "Terminal unlocked"
```

```
student@student:~/Unix$ sh 3b.sh
 Enter password:
 Confirm password:
 Terminal is locked
 Enter password to unlock:

 Invalid password
 Enter password again:

 Terminal unlocked
```

4a. Write a shell script that accepts one or more file names as arguments and convert all of them to uppercase, provided they exist in the current directory.

4a.sh

```
for file in "$@"
do
    if [ -f $file ]
    then
           ufile=`echo "$file" | tr "a-z" "A-Z"`
           if [ -f $ufile ]
           then
              echo "$ufile exists"
           else
              mv $file $ufile
           fi
    else
           echo "$file does not exist"
    fi
done
```

```
student@student:~/Unix$ touch a b
student@student:~/Unix$ sh 4a.sh a b c
 c does not exist
student@student:~/Unix$ touch a b
student@student:~/Unix$ sh 4a.sh a b c
 A exists
 B exists
 c does not exist
```

4b. Write a shell script that displays all the links to a file specified as the first argument to the script. The second argument, which is optional, can be used to specify in which the search is to begin. If this second argument is not present, the search is to begin in the current working directory. In either case, the starting directory as well as all its subdirectories at all levels must be searched. The script need not include any error checking.

4b.sh

```
file="$1"

if [ $# -eq 1 ]
then
        dir=.
else
        dir="$2"
fi

if [ -f "$file" ]
then
    set -- `ls -l "$file"`
    lcount="$2"

    if [ "$lcount" -eq 1 ]
    then
            echo "No other links"
    else
            set -- `ls -i "$file"`
            inode="$1"
            find "$dir" -xdev -inum "$inode" -print
    fi
else
        echo "File does not exist"
fi
```

```
student@student:~/Unix$ echo "This is a test file" > a.txt
student@student:~/Unix$ sh 4b.sh a.txt
 No other links
student@student:~/Unix$ ln a.txt /home/student/aa.txt
student@student:~/Unix$ sh 4b.sh a.txt /home/student
 /home/student/Unix/a.txt
 /home/student/aa.txt
```

5a. Write a shell script that accepts the filename as argument and displays its creation time if the file exists and if it does not send an output error message.

5a.sh

```
if [ -f "$1" ]
then
    echo "File Name : $1"
    echo "Creation Time : `stat "$1" | grep "Birth" |
    cut -d ' ' -f 3,4,5`"
else
    echo "File does not exist"
fi
```

```
student@student:~/Unix$ sh 5a.sh 4b.sh
 File Name : 4b.sh
 Creation Time : 2026-08-25 21:02:34.325449913 +0530
```

5b. Write a shell script to display the calendar for current month with current date replaced by *or** depending on whether the date has one digit or two digits.

5b.sh

```
set -- `date`
day="$3"

if [ $day -le 9 ]
then
    cal | sed "s/ $day / * /"
else
    cal | sed "s/ $day / ** /"
fi
```

```
student@student:~/Unix$ sh 5b.sh
     August 2026
 Su Mo Tu We Th Fr Sa
                    1
  2  3  4  5  6  7  8
  9 10 11 12 13 14 15
 16 17 18 19 20 21 22
 23 24 ** 26 27 28 29
 30 31
```

6a. Write a shell script to find a file/s that matches a pattern given as command line argument in the home directory, display the contents of the file and copy the file into the directory ~/mydir.

6a.sh

```
pattern="$1"
dest="$HOME/mydir"

if [ ! -d "$dest" ]
then
    mkdir -p "$dest"
fi

find "$HOME" -type f -name "$pattern" -print | while read file
do
    echo "File found: $file"
    echo "-------------------------"

    cat "$file"

    echo "-------------------------"
    cp "$file" "$dest"

    echo "$file copied to $dest"
done
```

```
student@student:~/Unix$ echo "Hello NMAMIT" > /home/student/wordzz.txt
 Hello NMAMIT
student@student:~/Unix$ sh 6a.sh "wordzz.txt"
 File found: /home/student/wordzz.txt
 -------------------------
 Hello NMAMIT
 -------------------------
 /home/student/wordzz.txt copied to /home/student/mydir
```

6b. Write a shell script to list all the files in a directory whose filename is at least 10 characters.(use expr command to check the length)

6b.sh

```
if [ -d "$1" ]
then
    for file in "$1"/*
    do
        if [ -f "$file" ]
        then
            bname=`basename "$file"`
            name="${bname%.*}"
            len=`expr length "$name"`

            if [ "$len" -ge 10 ]
            then
                echo "$bname"
            fi
        fi
    done
else
    echo "Directory does not exist"
fi
```

```
student@student:~/Unix$ sh 6b.sh /home/student
 10characterfile.txt
 duplicatedata.txt
```

${bname...}: Tells Bash to look at the value stored inside the variable $bname (e.g., document.txt).
% : Tells Bash to match and cut off text starting from the right side (the end) of the string.
.* : The pattern to look for. It matches a literal dot (.) followed by any characters ().

7a. Write a shell script that gets executed and displays the message either “Good Morning” or “Good Afternoon” or “Good Evening” or “Good Night” depending upon time at which the user logs in.

7a.sh

```
hour=` date +%H `

if [ $hour -ge 0 ] && [ $hour -lt 12 ]
then
        echo "Good Morning $LOGNAME"
elif [ $hour -ge 12 ] && [ $hour -lt 17 ]
then
        echo "Good Afternoon $LOGNAME"
elif [ $hour -ge 17 ] && [ $hour -lt 20 ]
then
        echo "Good Evening $LOGNAME"
else
        echo "Good Night $LOGNAME"
fi
```

```
student@student:~/Unix$  sh 7a.sh
Good Night student
```

7aa.sh

```
hour=` date +%H `

case $hour in
    0[0-9]|1[0-1])
        echo "Good Morning $LOGNAME"
        ;;
    1[2-6])
        echo "Good Afternoon $LOGNAME"
        ;;
    1[789])
        echo "Good Evening $LOGNAME"
        ;;
    2[0-3])
        echo "Good Night $LOGNAME"
        ;;
esac

# 0[0-9]|1[0-1] → 00–09 or 10–11
# 1[2-6] → 12–16
```

```
student@student:~/Unix$  sh 7a.sh
Good Night student
```

7b. Write a shell script that accepts a list of filenames as its argument, count and report occurrence of e a c h word that is present in the first argument file on other argument files.

7b.sh

```
if [ $# -lt 2 ]
then
    echo "Usage: 7b.sh wordfile.txt file1.txt file2.txt ..."
else
    for word in `cat "$1"`
    do
            for file in "$@"
            do
                if [ "$file" != "$1" ]
                then
                        count=`grep -iow "$word" "$file" | wc -l`
                        echo "$word : $file : $count"
                fi
            done
    done
fi
```

```
student@student:~/Unix$ cat words.txt
 hello
 world
 linux
student@student:~/Unix$cat file1.txt
 Hello world welcome to unix
student@student:~/Unix$ cat file2.txt
 hello students
 lets start the linux executions
 starting with hello world
student@student:~/Unix$ sh 7b.sh words.txt file1.txt file2.txt
 hello : file1.txt : 1
 hello : file2.txt : 2
 world : file1.txt : 1
 world : file2.txt : 1
 linux : file1.txt : 0
 linux : file2.txt : 1
```

8a. Write a shell script that determines the period for which a specified user is working on the system and display appropriate messages.

8a.sh

```
echo "Enter login name:"
read user

session=`loginctl list-sessions --no-legend | grep " $user " | head
-n 1`

if [ -z "$session" ]
then
        echo "$user is not logged in"
else

    sid=`echo "$session" | cut -d ' ' -f 1`

    login_time=`loginctl show-session "$sid" -p Timestamp --value`

    start=`date -d "$login_time" +%s`
    now=`date +%s`
    diff=`expr $now - $start`
    hours=`expr $diff / 3600`
    minutes=`expr \( $diff % 3600 \) / 60`
    echo "$user is working for $hours hours and $minutes minutes"
fi
```

```
student@student:~/Unix$ sh 8a.sh
 Enter login name:
 student
 student is working for 1 hours and 38 minutes
student@student:~/Unix$ sh 8a.sh
 Enter login name:
 shree
 shree is not logged in
```

8b. Write a shell script that reports the logging in of a specified user within one minute after he/she login. The script automatically terminates if the specified user does not login during a specified period of time.

8b.sh

```
echo -n "Enter login name: "
read user

echo -n "Enter time in minutes: "
read min

count=0
seconds=`expr $min \* 60`

while ! loginctl list-sessions --no-legend | grep -w "$user" > /dev/null
do
      sleep 5
      count=`expr $count + 5`
      echo "Waiting $user to login . . ."

      if [ $count -ge $seconds ]
      then
            echo "$user has not logged in within $min minutes"
            exit
      fi
done

echo "$user has logged in"
```

```
student@student:~/Unix$ sh 8b.sh
 Enter login name: student
 Enter time in minutes: 1
 student has logged in
student@student:~/Unix$ sh 8b.sh
 Enter login name: test
 Enter time in minutes: 1
 Waiting test to login . . .
 Waiting test to login . . .
 test has logged in
```

```
student@student:~/Unix$ su test
 Password :
test@student:~/Unix$
```

9a.Write a shell script that accepts the filename, starting and ending line number as an argument and displays all the lines between the given line number.

9a.sh

```
if [ $# -ne 3 ]
then
    echo "Invalid number of arguments"
    exit
fi

lines=`cat "$1" | wc -l `

if [ $2 -le 0 ] || [ $3 -le 0 ] || [ $2 -gt $3 ] ||
[ $3 -gt $lines ]
then
    echo "Invalid input"
    exit
fi

sed -n "$2,$3 p" "$1"
```

```
student@student:~/Unix$ cat>test.txt
 Line 1
 Line 2
 Line 3
 Line 4
 Line 5
 Line 6
 Line 7
student@student:~/Unix$ sh 9a.sh test.txt 3 5
 Line 3
 Line 4
 Line 5
```

9b. Write a shell script that folds long lines into 40 columns. Thus any line that exceeds 40 characters must be broken after the 40th , a "\" is to be appended as the indication of folding and the processing is to be continued with the residue. The input is to be supplied through a text file created by the user.

9b.sh

```
echo "Enter file name:"
read file

while read line
do
    len=`expr length "$line"`
    while [ $len -gt 40 ]
        do
            echo "`echo "$line" | cut -c 1-40 ` "'\'
            line=`echo "$line" | cut -c 41-`
            len=`expr length "$line"`
        done
        echo "$line"

done < "$file"
```

```
student@student:~/Unix$ cat file2.txt
 hello students
 lets start the linux executions, starting with hello world
student@student:~/Unix$ sh 9b.sh
 Enter file name:
 file2.txt
 hello students
 lets start the linux executions, startin\
 g with hello world
```

10a. Write an awk script that accepts date arguments in the form of dd-mm-yy and display it in the form month, day and year. The script should check the validity of the argument and in the case of error, display a suitable message.

10a.awk

```
{
    split($0, d, "-")

    if (d[1] < 1 || d[1] > 31 || d[2] < 1 || d[2] > 12)
    {
        print "Invalid date"
        exit
    }

    days[1]=31; days[2]=28;  days[3]=31;  days[4]=30;
    days[5]=31; days[6]=30;  days[7]=31;  days[8]=31;
    days[9]=30; days[10]=31; days[11]=30; days[12]=31;

    if (d[1] > days[d[2]+0])
    {
        print "Invalid date"
        exit
    }

    month[1]="January"; month[2]="February"; month[3]="March";
    month[4]="April";   month[5]="May";      month[6]="June";
    month[7]="July";    month[8]="August";   month[9]="September";
    month[10]="October"; month[11]="November"; month[12]="December";

    print month[d[2]+0] "," d[1] "," d[3]
    exit
}
```

```
student@student:~/Unix$ echo "30-02-2026" | awk -f 10a.awk
 Invalid date
student@student:~/Unix$ echo "18-08-2026" | awk -f 10a.awk
 August,18,2026
student@student:~/Unix$ awk -f 10a.awk
 18-09-2026
 September,18,2026
```

10b. Write an awk script to delete duplicated lines from a text file. The order of the original lines must remain unchanged.

10b.awk

```
{
    line[++n] = $0
}

END {
    for (i = 1; i <= n; i++)
    {
        flag = 1
        for (j = 1; j < i; j++)
        {
            if (line[i] == line[j])
                flag = 0
        }

        if (flag == 1)
        {
            print line[i]
            print line[i] > "out10b.txt"
        }
    }
}
```

10bb.awk

```
{
    if (!seen[$0])
    {
        print $0
        print $0 > "out10b.txt"
        seen[$0] = 1
    }
}
```

```
student@student:~/Unix$ cat>duplicate.txt
 Line 1
 Line 2
 Line 1
 Line 3
student@student:~/Unix$ awk -f 10b.awk duplicate.txt
 Line 1
 Line 2
 Line 3
student@student:~/Unix$ cat out10b.txt
 Line 1
 Line 2
 Line 3
```

11a. Write an awk script to find out total number of books sold in each discipline as well as total book sold using associate array down table as given below.

Books.txt

Electrical	34
Mechanical	67
Electrical	80
Computer_Science	43
Mechanical	65
Civil	98
Computer_Science	64

11a.awk

```
BEGIN {
    print"TOTAL NUMBER OF BOOOKS SOLD IN EACH CATAGORY"
}

{
    books [$1]+=$2
}

END {
    for(item in books)
    {
        printf("\t%-17s %1s %-5d\n", item, "=",books[item])
        total+=books[item]
    }
    printf("%-17s %1s %-5s\n", "Total Books Sold","=", total)
}
```

```
student@student:~/Unix$ cat Books.txt
 Electrical  34
 Mechanical  67
 Electrical  80
 Computer_Science    43
 Mechanical  65
 Civil   98
 Computer_Science    64
student@student:~/Unix$ awk -f 11a.awk Books.txt
 TOTAL NUMBER OF BOOKS SOLD IN EACH CATEGORY
        Civil             = 98
        Mechanical        = 132
        Electrical        = 114
        Computer_Science  = 107
 Total Books Sold  = 451
```

11aa.awk

```
BEGIN {
    print("TOTAL NUMBER OF BOOKS SOLD IN EACH CATEGORY")
}

{
    item = $1
    for (i = 2; i < NF; i++)
        item = item " " $i

    books[item] += $NF
}

END {
    for (item in books)
    {
        printf("\t%-17s = %-5d\n", item, books[item])
        total += books[item]
    }

    printf("\t%-17s = %-5d\n", "Total Books Sold", total)
}
```

```
student@student:~/Unix$ cat Books1.txt
 Electrical  34
 Mechanical  67
 Electrical  80
 Computer Science    43
 Mechanical  65
 Civil   98
 Computer Science    64
student@student:~/Unix$ awk -f 11aa.awk Books1.txt
 TOTAL NUMBER OF BOOKS SOLD IN EACH CATEGORY
        Civil             = 98
        Mechanical        = 132
        Electrical        = 114
        Computer Science  = 107
        Total Books Sold  = 451
```

When you use a loop like for (item in books), AWK does not sort the keys alphabetically, numerically, or by the order they appeared in the text file. Instead, it stores them internally using a mechanism called a hash table for speed.

To get a Sorted based on Key
awk -f 11aa.awk Books1.txt | sort

11b. Write an awk script to compute gross salary of an employee according to the rule given below.
If basic salary is <10000 then HRA=15% of basic &DA=45% of basic
If basic salary is>=10000 then HRA=20% of basic &DA=50% of basic.

11b.awk

```
BEGIN {
    FS=":"
}

{
    basic = $5
    if (basic < 10000)
    {
        hra = basic * 0.15
        da = basic * 0.45
    }
    else
    {
        hra = basic * 0.20
        da = basic * 0.50
    }
    gross = basic + hra + da

    print "Employee:", $2
    print "Basic Salary:", basic
    print "HRA:", hra
    print "DA:", da
    print "Gross Salary:", gross
    print "-----------------------------"
}
```

```
student@student:~/Unix$ cat>data.txt
 1:Shree:Programmer:ISE:8000
 2:Ramesh:Professor:ISE:12000
student@student:~/Unix$ awk -f 11b.awk data.txt
 Employee: Shree
 Basic Salary: 8000
 HRA: 1200
 DA: 3600
 Gross Salary: 12800
 -----------------------------
 Employee: Ramesh
 Basic Salary: 12000
 HRA: 2400
 DA: 6000
 Gross Salary: 20400
 -----------------------------
```